INF221 Assignment

Identification	
Group Number	1
Members	• Dan Trung Nguyen • Håkon Bråthen Børve • Milad Tootoonchi

Exercise 1

In [ ]:
```
'''
I chose to write my function such that it takes in the paper name aswell as the
I did this for the "fantasy" of the task solution, however one could have ignore
It is largly the same though either way.
'''

#1. h_index algorithm
def h_index(paper_list:list) -> int:
    '''
    this function determines the h-index of the reasearcher credited with the pa

    input:
        paper_list: a nested list of the names of the papers and the number of c
        List format: [[paper_name, citation_count], [paper_name, citation_count]

    output:
        int: the h-index of the researcher
    '''

    #sorting list according to citation count in descending order
    sorted_papers = sorted(paper_list, key=lambda x: x[1], reverse=True)

    #increasing h_index value until the citations do not equal or exceed the pap
    h_index_value = 0
    for idx, paper in enumerate(sorted_papers):
        if idx +1 <= paper[1]:
            h_index_value = idx + 1
        else:
            break
    return h_index_value
```

In [ ]:
```
#checking the algorithm
paper_list = [['paper1', 1], ['paper2', 8], ['paper3', 10], ['paper4', 3], ['pap
sorted_papers = sorted(paper_list, key=lambda x: x[1], reverse=True)
print(sorted_papers)
print(h_index(paper_list))
```

```
[['paper3', 10], ['paper9', 9], ['paper2', 8], ['paper8', 7], ['paper7', 6], ['pa
per6', 5], ['paper5', 4], ['paper4', 3], ['paper10', 2], ['paper1', 1]]
5
```

in juding runtime there are two parts of the alogrithm to consider, the sorting and the loop.

the sorting portion is O(n log n) for both worst case and avarage case the looping portion is linear looking only for a stopping point O(n) regardless of worst or avarage case

sorting dominates the looping and so the worst and avarage case are both O(n log n)

Exercise 2

In [51]:
```
#Node & Linked List Classes

#Node class
class Node:
    def __init__(self, value, prev=None, next=None):
        self._value = value
        self._prev = prev
        self._next = next

    def get_prev(self):
        return self._prev

    def set_prev(self, prev):
        self._prev = prev

    def get_next(self):
        return self._next

    def set_next(self, next):
        self._next = next

#DoubleLinkedList class
class doublelinkedlist:
    def __init__(self, head=None, tail=None):
        self._head = head
        self._size = 0

    def insert(self, value: Node):
        if self._size == 0:
            self._head = value
            self._size += 1
        else:
            value.set_next(self._head)
            self._head.set_prev(value)
            self._head = value
            self._size += 1

    def display(self):
        print("List: ")
        current = self._head
        while current is not None:
            print(current._value, end=" ")
            current = current.get_next()
        print()
```

In [52]:
```
# Making the list
L = doublelinkedlist()
A = Node("A")
B = Node("B")
C = Node("C")

L.insert(C)
L.insert(B)
L.insert(A)
```

In [ ]:
```
#Function to insert

def arbitrary_Insert(prev:Node, value:Node, linklist:doublelinkedlist):
    '''
    this function inserts a Node in a double linked list at an arbitrary positio

    input:
        prev: the node the new node should be inserted after
        value: the Node to be inserted
        list: the double linked list where the node should be inserted
    '''
    #inserting node
    if prev is None:
        value.set_next(linklist._head)
        if linklist._head is not None:
            linklist._head.set_prev(value)
        next = linklist._head
        linklist._head = value
    else:
        value.set_next(prev.get_next())
        next = prev.get_next()
        value.set_prev(prev)

    #adjusting sourounding nodes
    if next is not None:
        next.set_prev(value)
    if prev is not None:
        prev.set_next(value)

B2 = Node("B2")
arbitrary_Insert(C, B2, L)
L.display()
```

```
List:
A B C B2
```

Psuedocode

ALGORITHM InsertAfter(prev, value, L)

```
    IF prev = NIL THEN
        value.next ← L.head

        IF L.head ≠ NIL THEN
            L.head.prev ← value

        L.head ← value
        value.prev ← NIL
```

```
    ELSE
        value.next ← prev.next
        value.prev ← prev

        IF prev.next ≠ NIL THEN
            prev.next.prev ← value

        prev.next ← value

END
```

Runtime of the modified algorithm is O(1) as it is only a single reshuffeling of values and no loops of any sort

Exercise 3

Recall that a singly linked list is a list where each element will point towards its successor, unlike the doubly linked list where it points towards its predecessor and successor. If we want to delete an element from such a list, we first have to search for the key k in it. This can be done by simply going through every term, one by one until we find our element. This searching method will scale on our list length n, and on average the searching will go about half before it finds the element with key k. The singly linked list will take the form of:

`[x x.next] -> [x x.next] -> [x x.next]`

Since it does not link to its predecessor, we need to store it for each search. This is because we need to close the "hole" after deleting the key k. With this in mind, here is an example of such an algorthm: (L is the name of the list)

```
    Delete-Singly(L, k)
        x = L.head
        x_prev = NIL

        # Start by searching for the key
        while x != NIL and x.key != k
            x_prev = x
            x = x.next

            if x == NIL
                return "Key is NIL"

        # If x.key in list, then make current x.prev and x.key into
    previous

        if x != L.head
            x_prev.next = x.next

        else
            L.head = x.next
```

On average, the searching will find the element half into the list. This means that we've gone through $\frac{n}{2}$ elements. We can write this as $O(\frac{n}{2})$, or simplify this to $O(n)$. If the element is not found, meaning it goes to the end of the list and meets `NIL`, then it would've gone through the whole list. Since a failed search means the algorhtm has to always go through n times, the upper bound and lower bound are the same, meaning the total runtime would be $\Theta(n)$.

After searching, going to the previous element and changing its properties is constant. This means that this is not affected by list length. Meaning we can write it is $O(n)$. Combining these runtimes together gives us:

$$O(n) + O(1) = O(n+1) = O(n)$$

If our key is the first element of the list, the runtime also becomes constant as we immediately find it.

$$O(1) + O(1) = O(2) = O(1)$$

Exercise 4

1.

Instead of shifting everything, only move elements whose probe sequence would be broken.

```
    DELETE(key):
        i = 0
        while i < m:
            pos = h(key, i)

            if table[pos] == EMPTY:
                return NOT_FOUND

            if table[pos] != DELETED and table[pos].key == key:
                table[pos] = DELETED
                return SUCCESS

            i = i + 1

        return NOT_FOUND
```

2.

Hash function:

$$h(k,i) = (h'(k) + (-1)^i * i^2) \mod 11,$$
$$h'(k) = k \mod 11$$

Inserted in order:

- (0,A): 0 -> place at 0
- (2,B): 2 -> 2
- (1,C): 1 -> 1
- (0,D):
  - i=0 -> 0 (occupied)
  - i=1 -> 0−1=10 -> 10
- (3,E): 3 -> 3
- (5,F): 5 -> 5
- (2,G):
  - i=0 -> 2 (occupied)
  - i=1 -> 1 (occupied)
  - i=2 -> 2+4=6 -> 6
- (6,H):
  - i=0 -> 6 (occupied)
  - i=1 -> 5 (occupied)
  - i=2 -> 10 (occupied)
  - i=3 -> 6−9=8 -> 8

Hash Table

Index	Entry
0	A
1	C
2	B
3	E
4	-
5	F
6	G
7	-
8	H
9	-
10	D

Exercise 5

sort intervals by start time

```
    active = empty list (or min-heap by end time)

    for (a, b) in intervals:
        # Remove non-overlapping intervals
        remove all (a', b') from active where b' <= a
```

```
        # Report overlaps
        for (a', b') in active:
            report (a', b') overlaps with (a, b)

        # Add current interval
        active.add((a, b))
```

In [ ]:
```python
def find_overlaps(intervals):
    intervals.sort(key=lambda x: x[0])

    active = []  # will store active intervals
    overlaps = []

    for current in intervals:
        a, b = current
        active = [iv for iv in active if iv[1] > a]

        for iv in active:
            overlaps.append((iv, current))
        active.append(current)

    return overlaps


# Example
L = [(2, 5), (6, 9), (7, 11), (10, 15), (12, 14), (13, 17), (19, 21)]

result = find_overlaps(L)

for x, y in result:
    print(f"{x} overlaps with {y}")
```

```
(6, 9) overlaps with (7, 11)
(7, 11) overlaps with (10, 15)
(10, 15) overlaps with (12, 14)
(10, 15) overlaps with (13, 17)
(12, 14) overlaps with (13, 17)
```

The runtime is O(n log(n + k)), where n is the number of intervals and k is the number of overlapping pairs reported. First, the intervals are sorted by start time, which takes O(n * log(n)). Then, we process each interval once while maintaining a set of active intervals. Using a heap, each interval is inserted and removed at most once, giving a total cost of O(n * log(n)).

Instead of comparing all pairs (which would take O(n^2)), the algorithm only compares intervals that are active at the same time. Each overlapping pair is reported exactly once, contributing O(k) to the runtime. Therefore, the total runtime becomes O(n * log(n + k)). This is optimal, since sorting already requires O(n * log(n)), and reporting all overlaps necessarily takes O(k).